

LeetCode 304: Range Sum Query 2D - Immutable

2D Array Prefix Questions • Concept & Implementation

SAMSUNG

NVIDIA

Core Concept: Using a 2D prefix sum array allows us to efficiently calculate the sum of any submatrix in $O(1)$ time after an $O(N \times M)$ preprocessing step.

1. Building the 2D Prefix Array

To construct the prefix sum array *pref*, we iteratively add the values from the original matrix and the previously computed prefix sums.

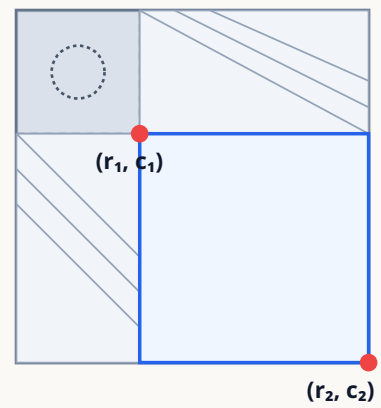
- Start with the current cell value: $pref[i][j] = matrix[i][j]$
- Add the sum from the row above: $+ pref[i-1][j]$
- Add the sum from the column to the left: $+ pref[i][j-1]$
- **Crucial Step:** Subtract the top-left diagonal $pref[i-1][j-1]$ because it was **added twice** in the previous two steps!

2. Using the 2D Prefix Array (Submatrix Sum)

To find the sum of a submatrix bounded by top-left (r_1, c_1) and bottom-right (r_2, c_2) :

- Start with the total sum up to the bottom-right corner: $ans = pref[r_2][c_2]$
- Subtract the region above the submatrix: $- pref[r_1-1][c_2]$

- Subtract the region to the left of the submatrix: - $\text{pref}[r_2][c_1-1]$
- **Correction:** The top-left region $\text{pref}[r_1-1][c_1-1]$ was subtracted twice, so we must add it back!



3. C++ Implementation

```

class NumMatrix {
public:
    vector<vector<int>> pref;

    NumMatrix(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        pref.resize(n, vector<int>(m));

        for(int i = 0; i < n; i++){
            for(int j = 0; j < m; j++){
                pref[i][j] = matrix[i][j];
                if(i > 0){
                    pref[i][j] += pref[i-1][j];
                }
                if(j > 0){
                    pref[i][j] += pref[i][j-1];
                }
                if(i > 0 && j > 0){
                    pref[i][j] -= pref[i-1][j-1];
                }
            }
        }
    }

    int sumRegion(int row1, int col1, int row2, int col2) {
        int ans = pref[row2][col2];

        if(row1 > 0){
            ans -= pref[row1-1][col2];
        }
        if(col1 > 0){
            ans -= pref[row2][col1-1];
        }
        if(row1 > 0 && col1 > 0){
            ans += pref[row1-1][col1-1];
        }

        return ans;
    }
};

/**
 * Your NumMatrix object will be instantiated and called as such:
 * NumMatrix* obj = new NumMatrix(matrix);
 * int param_1 = obj->sumRegion(row1,col1,row2,col2);
 */

```

